# SoK: Understanding zkVM: From Research to Practice

Yunbo Yang*
The State Key Laboratory of Blockchain and Data Security, Zhejiang University
Hangzhou High-Tech Zone (Bin jiang) Institute of Blockchain and Data Security
Hangzhou, China
yyb9882@gmail.com

Yuejia Cheng*
University of Sussex
Hangzhou, China
chengyuejia@foxmail.com

Haibo Tang
The State Key Laboratory of Blockchain and Data Security, Zhejiang University
Hangzhou High-Tech Zone (Bin jiang) Institute of Blockchain and Data Security
Hangzhou, China
hbtang001@163.com

Guomin Yang
School of Computing and Information Systems, Singapore Management University
Singapore, Singapore
gmyang@smu.edu.sg

Bingsheng Zhang†
The State Key Laboratory of Blockchain and Data Security, Zhejiang University
Hangzhou, China
bingsheng@zju.edu.cn

Kui Ren†
The State Key Laboratory of Blockchain and Data Security, Zhejiang University
Hangzhou, China
kuiren@zju.edu.cn

## Abstract

Zero-knowledge virtual machine (zkVM) is a powerful infrastructure for proving the correctness of a program execution with a succinct proof, attracting significant interest from researchers, developers, and users. It has been widely used in applications such as blockchain rollups, privacy-preserving machine learning, and off-chain computation. As the field grows, a wide range of zkVMs have been proposed. However, they adopt different choices in instruction formats, trace layouts, and proving backends, which results in a highly heterogeneous design landscape and makes it difficult to understand the relations among these systems.

To bridge this gap, we provide a comprehensive study of zkVMs that covers both their theoretical foundations and practical implementations. We decompose zkVMs into three layers: (1) the ISA layer, which defines instruction semantics and determines the structure of the execution trace, (2) the VM layer, which captures program execution and organizes constraints through modular circuit components, and (3) the proving layer, which converts execution traces into algebraic constraints and generates the final proofs. This decomposition allows us to isolate the role of each layer while also examining how they interact in real systems. To give readers a more direct understanding of how these design choices affect performance, scalability, and usability, we conduct a comprehensive experimental evaluation of representative zkVMs following this layered framework. Finally, we conclude the paper by summarizing the main observations from our analysis and outlining several potential directions for zkVM design and implementation.

This work will appear in AsiaCCS 2026.

## CCS Concepts

• **Security and privacy** → *Cryptographic protocols.*

## Keywords

Zero-knowledge Proof, Zero-knowledge Virtual Machine

## 1 Introduction

Imagine a world where programs can prove their own correctness without revealing how they run. From zero-knowledge proofs (ZKPs) [1, 2] to zero-knowledge succinct non-interactive argument of knowledge (zkSNARK) [3, 4], this has gradually evolved into reality. However, building verifiable computation systems for general-purpose programs remains complex. Developers must manually encode instruction semantics into circuits, handle field arithmetic, and integrate cryptographic proof systems. To simplify this process, zero-knowledge virtual machines (zkVMs) have emerged as a powerful abstraction layer, which automatically transform a program execution trace into a succinct zero-knowledge proof.

The motivation behind zkVMs lies in bridging the usability gap between cryptographic proofs and real-world computation. Traditional zk-SNARKs [5, 6] focus on proving the correctness of circuits, whereas zkVMs elevate this abstraction to the level of ISAs, enabling developers to prove the correctness of entire programs. This new paradigm separates proof generation from circuit design, allowing anyone without a strong cryptography background to have access to verifiable computation. Over the past few years, many zkVMs such as Risc0 [7, 8], SP1 [9, 10], Jolt [11], Scroll [12], and OpenVM [13] have been proposed, each has different trade-offs among performance, scalability, transparency, and modularity. Different projects use distinct instruction formats, trace layouts, and proving backends, and these incompatibilities make systematic evaluation and fair comparison difficult. To understand the zkVM ecosystem, we ask the following research questions:

- **RQ1:** How can we construct a unified model that decomposes zkVMs into fundamental components across execution, arithmetization, and proof layers?
- **RQ2:** What are the key design principles, trade-offs, and optimization strategies used in existing zkVMs?

*These authors contributed equally to this work.
†Corresponding Authors.

- **RQ3:** How can we benchmark zkVM implementations across different architectures to identify efficiency and usability bottlenecks?

**Our Work.** To address these questions, we conduct a systematic study of modern zkVMs. We first decompose zkVMs into three fundamental layers, including instruction set architecture (ISA), virtual machine (VM) architecture, and the proving layer. The ISA layer defines the semantics of computation and determines how high-level programs are translated into low-level instructions. The VM architecture layer captures how these instructions are executed and recorded into execution trace tables, and then uses constraints to prove the correctness. Finally, the proof layer uses a zero-knowledge proof system to generate cryptographic proofs to show the correctness of program execution.

Based on this decomposition, we analyze representative zkVM implementations to understand how different systems instantiate and optimize each layer. We further perform a systematic benchmark to evaluate their efficiency, scalability, and usability, providing insights on how architectural and cryptographic design choices impact performance. Our goal is to present a unified framework that clarifies the internal composition of zkVMs, highlights design trade-offs, and offers guidance for building scalable, interoperable, and developer-friendly zkVM infrastructures.

**Summary of Contributions.** We summarize our contributions as follows:

- **Unified Architectural zkVM Framework.** We present a comprehensive three-layer decomposition of zkVMs, including the ISA, VM, and proving layers. This structure systematizes the end-to-end verifiable computation pipeline and clarifies the roles. In addition, building on this framework, we further categorize representative zkVM systems according to these three structural dimensions.
- **Cross-Layer Interaction Analysis.** We identify and quantify how design choices propagate across layers—revealing that ISA granularity determines trace geometry, VM circuit structure influences PIOP backend selection, and field choice constrains recursive composition strategies.
- **Comprehensive Experimental Evaluation.** We conduct a comprehensive benchmark across state-of-the-art zkVMs to evaluate performance, scalability, and usability, offering practical insights into the tradeoffs between performance and scalability.

**Related Work.** Prior studies on zero-knowledge systems fall into two categories. First, surveys on zk-SNARK constructions and theoretical applications. For example, Lavin et al. [14] survey foundational components such as zkVMs, DSLs, libraries, and general-purpose frameworks. Liang et al. [15] provide a classification of more than forty SNARK systems and eleven libraries, focusing mainly on the proving-system layer. These works offer valuable overviews of the broader ZKP ecosystem but do not analyze the architectural design space or system-level tradeoffs specific to zkVMs.

The second category directly studies targeted zkVM implementations. For example, the thesis by Velička [16] compares RISC0 and SP1 from architectural and performance perspectives, while Hochrainer et al. [17] investigate soundness issues through systematic fuzzing across several real-world zkVMs. Gassmann et al. [18] evaluate compiler optimizations and their impact on zkVM performance. These analyses offer insights into particular components of zkVM design, but they do not provide a unified architectural synthesis for zkVMs.

## 2 Preliminaries

### 2.1 Zero-knowledge Virtual Machine

zkVM executes a program on a specified virtual machine and proves that the final execution trace is consistent with the VM transition rules. An executor first runs the program and inputs to produce a deterministic execution trace (registers, memory, I/O events). This trace is then arithmetized into constraints (e.g., AIR/PLONKish constraints), and a proof is generated using a zkSNARK protocol as a backend. The proof shows the correctness of state transitions, including instruction semantics, memory consistency, and system events. The verifier checks the succinct proof without re-executing the entire program. In addition, the zero-knowledge property is optional and depends on the chosen backend and configuration.

### 2.2 Polynomial Commitment Scheme (PCS)

A Polynomial Commitment Scheme (PCS) [19–21] allows a prover to commit to a hidden polynomial and later reveal its evaluation at chosen points along with an opening proof. Once the commitment has been published, it becomes computationally infeasible for the prover to tamper with the underlying polynomial. Formally, a PCS is a tuple of algorithms PCS = (Setup, Commit, Open, Verify) described as follows:

- $\mathsf{Setup}(1^\lambda) \to \mathsf{srs}$: On input of the security parameter $\lambda$, it generates a structured reference string srs.
- $\mathsf{Commit}(\mathsf{srs}, p) \to cm$: Outputs a public commitment $cm$ to a private polynomial $p(X)$.
- $\mathsf{Open}(\mathsf{srs}, p, cm, z) \to (v, \pi)$: Given an evaluation point $z$, it produces the claimed value $v = p(z)$ together with an opening proof $\pi$.
- $\mathsf{Verify}(\mathsf{srs}, \pi, cm, v, z) \to \mathsf{accept/reject}$: Checks if $(v, \pi)$ is a valid opening of $cm$ at point $z$.

A secure PCS satisfies the following standard properties:

- **Completeness:** An honest prover with a valid commitment to $p(X)$ can always produce an opening proof $\pi$ that the verifier accepts.
- **Knowledge Soundness:** Any prover convincing the verifier must implicitly define a unique polynomial $\tilde{p}(X)$ consistent with the reported openings.
- **Zero-Knowledge:** There exists an efficient simulator that can generate transcripts indistinguishable from real proofs without access to $p(X)$.

### 2.3 Zero-knowledge Succinct Non-interactive Argument of Knowledge (zkSNARK)

Zero-knowledge Succinct Non-interactive Argument of Knowledge (zkSNARK), such as Marlin [22] and Plonk [5], allows a prover to convince a verifier that a given relation is true with a succinct proof without leaking any additional information. The prover does not interact with the verifier in the protocol execution. Formally, a zkSNARK is composed of three primary algorithms:

- Setup$(1^\lambda) \rightarrow$ srs: Given the security parameter $\lambda$, this algorithm produces a universal structured reference string srs.
- Prove$(\text{srs}, C, \vec{x}; \vec{w}) \rightarrow \pi$: Using srs, a circuit $C$, public inputs $\vec{x}$, and private witness $\vec{w}$, the prover generates a proof $\pi$.
- Verify$(\text{srs}, \pi, \vec{x}) \rightarrow$ accept/reject: The verifier checks whether $\pi$ correctly demonstrates that $C(\vec{x}, \vec{w}) = 0$ holds.

When the circuit $C$ is evaluated on inputs $(\vec{x}, \vec{w})$, the witness $\vec{w}$ is internally extended to include all intermediate wire values, forming an extended witness $\hat{\vec{w}}$. A sound universal zkSNARK must satisfy three fundamental properties:

- **Completeness:** An honest prover always convinces an honest verifier.
- **Knowledge Soundness:** Any accepted proof implies the existence of an efficiently extractable witness with non-negligible probability.
- **Zero-Knowledge:** The proof reveals no private information beyond the validity of the statement.

**Design Paradigm of Universal zkSNARKs.** As observed by Bünz et al. [23], a universal zkSNARK can be conceptually decomposed into three components: a Polynomial Interactive Oracle Proof (PIOP) [24], a Polynomial Commitment Scheme (PCS) [19], and the Fiat–Shamir transformation [2].

Within the PIOP layer, the prover shows that the instance–witness pair satisfies all constraints by constructing prover polynomials, while the verifier only obtains oracle access to these polynomials. The PIOP layer is purely algebraic. Subsequently, the polynomial oracles are instantiated through a PCS [19, 25, 26], where oracle access is replaced with polynomial commitments and openings. Concretely, the prover first commits to the relevant polynomials, responds with evaluations and opening proofs based on verifier challenges [27, 28], and the verifier checks their validity. Finally, the Fiat–Shamir transformation [2] converts the interactive protocol into a fully non-interactive proof system.

## 3 Overview

The zkVM workflow begins with a program and input $(x, w)$, where $x$ is the public instance and $w$ is the private witness. This workflow as shown in Figure 1 can be decomposed into the following steps.

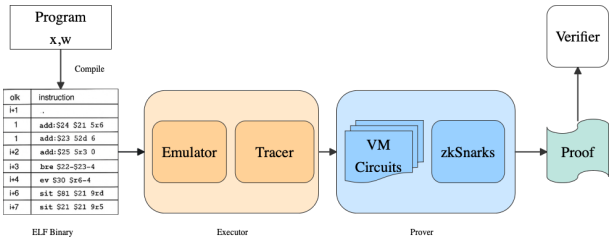


**Figure 1: Workflow of Modern zkVM**

(1) **Program Compilation.** The program is first compiled into an ELF (Executable and Linkable Format) binary, which is a list of low-level instructions, and then it will be executed and proved by the zkVM.

(2) **Execution And Trace Generation.** The compiled instructions together with input $(x, w)$ are executed by the VM (virtual machine), and it will be recorded as full execution traces for proof generation, including CPU operations, ALU operations and memory accesses that occur during program execution.

(3) **Constraints Construction.** The execution traces are verified by a set of VM circuits that impose algebraic constraints ensuring the correctness of each step in the program execution. These constrained traces then serve as the inputs to the zero-knowledge proving system.

(4) **Proof Generation.** The prover generates a succinct proof using a zk proof system, which shows the correct execution of the compiled program on input $(x, w)$.

(5) **Verification.** The proof is then sent to an external verifier. It verifies the proof to check the correctness of program execution without re-executing it.

Then, the above-mentioned steps can be summarized with the following algorithm interfaces.

- Compile(path)→ elf. This step involves translating a high-level program (e.g., C++, Rust) into an executable format, specifically the ELF.
  - Input. The path path to the source code of the guest program.
  - Output. An ELF file .elf, which is an executable and linkable format of program.
- Execute(elf, private_inputs) → (traces, public_inputs). This step is responsible for executing a user program and tracing important events which occur during execution (i.e., memory reads, alu operations, etc) within the VM(virtual machine), using the provided private inputs.
  - Input. The compiled ELF file elf, and some private data private_inputs.
  - Output. The results public_inputs of the computation that can be shared publicly without revealing the private inputs. And the execution traces of the ELF file.
- Prove(pk, traces) → proof. This step generates a proof to show the correctness of the computation performed by the zkVM.
  - Input. The proving key pk, execution traces traces. The traces include the public inputs public_inputs and private inputs private_inputs.
  - Output. A cryptographic proof proof that the computation was performed correctly.
- Verify(vk, proof, public_inputs) → bool. This step verifies the proof to ensure that the computation was performed correctly.
  - Input. The verify key vk, a cryptographic proof proof, and the public data public_inputs.
  - Output. A boolean value (true or false) indicating whether the proof is valid.

Next, we give a three-layer architecture of zkVM as shown in Figure 2. Based on this decomposition, we then categorize the zkVM systems in Table 1 along three architectural components: ISA, VM that produces execution traces, and the prover that generates the final proofs. This categorization offers a structured view of how

| | Basic | ISA Layer | | Execution Layer | | | VM Proving Layer | | | Aggregation | | SNARK Wrapping | | Maturity | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| zkVM | Ver. | ISA | Precompiles | Arch | Trace | Mem Model | Prover | Field | Mem Check | Strategy | PCS | Scheme | Field | Tools | Docs | Status | Ref |
| SP1 | v5.2.2 | rv32im | SHA2,Keccak,ECC,BLS | Harvard | Segment | Reg+RAM | Plonky3 | BabyBear | Off:Perm(GP) | Rec→Wrap | FRI | Groth16 | BN254 | ● | ● | Prod | [9, 10] |
| RISC Zero | v3.0.3 | rv32im | SHA2,Keccak,ECC | VonNeu | Segment | Paged(1KB) | Custom | BabyBear | Hyb:Pag+Perm | Recursive | FRI | Groth16 | BN254 | ● | ● | Prod | [7, 8] |
| OpenVM | v1.4.0 | rv32im | SHA2,Keccak,ECC | Harvard | Modular | Reg+RAM | Plonky3 | BabyBear | Off:LogUp | Recursive | FRI | Plonk | BN254 | ○ | ○ | Prod | [13] |
| Pico | v1.1.6 | rv32im | SHA2,Keccak | Harvard | Segment | Reg+RAM | Plonky3 | BabyBear | Off:Perm(GP) | Recursive | FRI | Groth16 | BN254 | ○ | ○ | Alpha | [29] |
| Ceno | master | rv32im | Limited | Harvard | Non-unif | Reg+RAM | GKR | Goldilocks | Off:GKR Sum | Rec GKR | Sumchk | - | - | ○ | ○ | Alpha | [30] |
| Airbender | v0.5.0 | rv32im | Limited | Harvard | - | Reg+RAM | STARK | Goldilocks | Off:Perm | Recursive | FRI | - | - | ○ | ○ | Alpha | [31] |
| Jolt | v0.3.0 | rv32i | Limited | Harvard | Lookup | Unified | Lasso | BN254 | Off:Spice | Sumchk Rec | HyKZG | - | - | ○ | ○ | Alpha | [11, 32, 33] |
| Nexus | v0.3.4 | rv32i | Limited | Harvard | Stwo | Reg+RAM | Stwo | M31 | Off:LogUp | - | C-FRI | - | - | ○ | ○ | Alpha | [34, 35] |
| ZisK | v0.10.0 | rv64ima | Limited | Harvard | PIL | Reg+RAM | PIL2 | Goldilocks | Off:Plookup | VADCOP | FRI | - | - | ○ | ○ | Alpha | [36] |
| Cairo | v2.8.5 | Cairo | Pedersen,ECDSA,Range | Harvard | AIR | Write-once | Stwo | M31 | Off:LogUp | SHARP Rec | C-FRI | Native | - | ● | ● | Prod | [37, 38] |
| Cairo-M | v0.1.0 | Cairo-M | Limited | Harvard | - | Write-once | Stwo | M31 | Off:LogUp | - | C-FRI | - | - | ○ | ○ | Early | [39] |
| Miden | v0.11 | Miden | Poseidon,Rescue | Stack | AIR | Op Stack | STARK | Goldilocks | Off:Chiplets | Recursive | FRI | - | - | ○ | ○ | Beta | [40] |
| Valida | v1.0.0+ | Valida | Limited | Harvard | Segment | Reg+RAM | STARK | Goldilocks | Off:Perm | - | FRI | - | - | ○ | ○ | Alpha | [41] |
| snarkVM | v0.16.0 | Leo/Aleo | Poseidon,BHP,BLS | - | R1CS | Circuit | Marlin | BLS12-377 | Off:R1CS | - | KZG | Marlin | BLS12 | ○ | ● | Prod | [42] |
| Lean | - | Custom | None | Harvard | - | - | WHIR | KoalaBear | Off:LogUp | - | WHIR | - | - | ○ | ○ | Early | [43] |
| Ziren | v1.2.1 | MIPS32 | SHA2,Keccak,BN254 | Harvard | Segment | Reg+RAM | Plonky3 | KoalaBear | Off:Perm(GP) | Rec→Wrap | FRI | Groth16 | BN254 | ○ | ○ | Beta | [44] |
| o1vm | - | MIPS32 | None | VonNeu | - | - | Kimchi | Pasta | Off:Custom | IVC Fold | IPA | - | - | ○ | ○ | Alpha | [45] |
| zkWasm | - | WASM | Limited | Stack | Halo2 | Linear | Halo2 | BN254 | Off:EID Intv | - | KZG | Halo2 | BN254 | ○ | ○ | Alpha | [46] |
| Novanet | - | WASM | None | - | - | - | Nova | Pasta | Off:IVC Fold | Nova Fold | IPA | - | - | ○ | ○ | Early | [47, 48] |
| Powdr | v0.1.3 | Multi | Limited | Config | - | Config | Plonky3 | Goldilocks | Off:Config | - | FRI | Halo2 | BN254 | ○ | ○ | Alpha | [49] |

**Table 1: Comprehensive Comparison of zkVM Architectures (Maturity: ●=Excellent; ○=Good; ○=Limited.)**

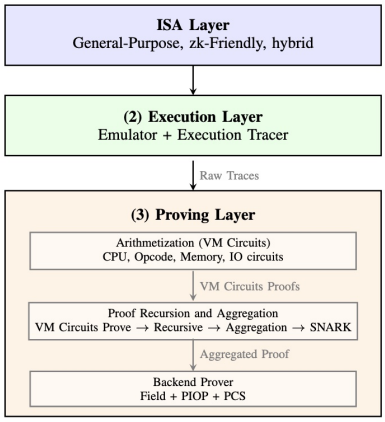


**Figure 2: Three-Layer Architecture of zkVM**

modern zkVMs are built. Based on this structure, we then provide a high-level overview of these three components to clarify their roles and the design choices behind them.

**(1) Instruction Set Architecture.** In zkVMs, the instruction set architecture (ISA) serves as a foundational component that dictates the structure and behavior of computations within the virtual machine. zkVM ISAs can be categorized into three main paradigms: General-Purpose ISAs, zk-Friendly ISAs, and Hybrid ISAs, each with distinct trade-offs illustrated as follows.

*General-Purpose ISA.* General-Purpose ISAs are designed to execute various applications and are typically used in traditional computing environments. In the context of zkVMs, these ISAs, such as RISC-V [50, 51] and MIPS [52], are adapted to facilitate the execution of arbitrary computations while generating zero-knowledge proofs.

*zk-Friendly ISA.* zk-Friendly ISAs are specifically designed or adapted to optimize the efficiency of zero-knowledge proof generation. These ISAs, such as Cairo [38] and Valida [41], simplify operations and align closely with the requirements of proof systems. However, they suffer from limited capabilities.

*Hybrid ISA.* Hybrid ISAs, such as SP1 [9], combine elements from both general-purpose and zk-friendly ISAs, aiming to balance flexibility with optimization for proof generation. These ISAs have a tradeoff between the broad applicability of general-purpose ISAs and the efficiency of zk-friendly ISAs. The choice of ISA in a zkVM significantly influences its performance and applicability. We provide a detailed comparison in Section 4.

**(2) Execution Layer.** The execution layer runs the compiled program and produces the exeuction traces. Existing zkVMs separate the program execution into two distinct roles: an emulator that realizes the virtual machine semantics, and an execution tracer that records the state transitions generated during execution.

- **Emulator.** The emulator implements the target ISA runtime and provides the concrete execution environment for the program. It interprets instructions, updates registers and memory, and maintains the architectural state required by the VM.
- **Execution Tracer.** The execution tracer records how a program runs, including decoded instructions, arithmetic results, and memory reads or writes. It connects the emulator to the proving layer, balancing accuracy, efficiency, and modularity to achieve scalable zkVMs. Its design is key to zkVM performance, since the size and structure of the traces directly affect constraint complexity and proving time.

In most zkVMs currently in use, like SP1 [9] and RISC Zero [7], they utilize Von Neumann architecture [51] in their emulator, in which instructions and data share the same logical memory space. But when generating proof, these execution traces are usually arranged in a Harvard-style configuration, isolating instruction fetches from data memory accesses. This separation can also alleviate constraint coupling and simplify lookup relations in the proving circuits [28, 33]. For example, in SP1, program memory is controlled by the Program Chip and data memory is controlled by the Memory Chip, even though both originate from the same RISC-V address space inside the emulator. This hybrid design- Von Neumann semantics and Harvard-style circuit decomposition—has

gained acceptance for common application within zkVM architectures.

**(3) Proving Layer.** The proving layer transforms execution traces into a succinct cryptographic proof. It consists of two main components: arithmetization that encodes traces into several algebraic constraints, and the backend prover that commits to polynomials and generates proofs.

*Arithmetization (VM Circuits).* In a zkVM, the execution trace acts as the witness, while the VM circuits define the algebraic relations that ensure correct instruction execution, state transitions, and memory consistency. These relations are arithmetized into circuits that the prover should satisfy during proof generation. In modular zkVM architectures such as SP1, Nexus, and Valida, the overall VM circuit is further decomposed into specialized sub-circuits described as follows.

- **CPU circuits.** Encode the core arithmetic logic of the virtual machine. They integrate opcode, register, and pc updates, ensuring that each instruction's effects are correctly reflected in the machine state.
- **Opcode circuits.** Capture per-instruction semantics, usually grouped by functionality (e.g., ALU, branch, load/store, system). In SP1, each opcode corresponds to a distinct "Chip," linked by cross-table lookups.
- **Memory circuits.** Enforce read–write consistency through multiset, permutation checks, ensuring every read returns the latest written value.
- **I/O and precompile circuits.** Implement non-ISA primitives such as hash functions, elliptic-curve arithmetic, and system precompiles. These specialized gadgets extend the base ISA to accelerate cryptographic workloads.
- **Other circuits.** Including the global invariants across cycles constraints, consistent transitions of PC, registers, flags, and opcode-ordering constraints between cycles.

VM circuits constrain the execution traces to ensure that they represent a valid program running under a target ISA. This separation between trace generation (execution layer) and arithmetization (proving layer) enables zkVMs to scale efficiently, support recursive proof composition, and handle complex workloads.

*Proof Aggregation and Recursion.* In practical settings, program executions often produce traces that are too large to be accommodated within a single proving round. Modern zkVMs address this through a multi-stage recursive proof pipeline. First, the trace is partitioned into segments, and each segment is proven independently using a fast prover optimized for throughput (e.g., FRI-based STARKs over small fields like BabyBear). Second, these segment proofs are recursively aggregated through one or more layers, where a verifier circuit checks and combines proofs until a single root proof remains. Finally, for on-chain deployment, many zkVMs apply a SNARK wrapping step, in which the aggregated STARK proof is verified inside a SNARK circuit (typically Groth16 over BN254), producing a constant-size proof (~200 bytes) suitable for cheap verification on Ethereum. Systems like SP1 and RISC Zero adopt this multi-stage strategy. This architecture enables parallelism at the segment level, reduces peak memory usage, and decouples prover efficiency from on-chain verification cost. Notably, the stages in the proving pipeline target distinct performance criteria. Segment proving maximizes throughput by leveraging small-field arithmetic and FRI commitments. Recursive aggregation is engineered for recursion compatibility, whereas the on-chain wrapper minimizes proof size and verification overhead, often through pairing-based SNARK constructions like Groth16.

*Backend Prover.* The backend prover commits to the constrained polynomials and generates cryptographic proofs to show the correctness of program execution. The performance of the backend prover is typically evaluated by the following metrics:

- **Constraint throughput** measures how many constraints can be generated and proven per second.
- **Proof size and verification time** respectively represent the total amount of data included in the proof and the time that a verifier checks a proof, which affects bandwidth and on-chain storage cost.
- **Memory consumption** captures the amount of memory used during the proof generation phase.

These metrics are determined by the design choices of the proof system, including the field that defines the arithmetic environment, the PIOP that determines constraint encoding and interaction pattern, and the PCS that governs how commitments and openings are performed. Different combinations of these components lead to distinct trade-offs between prover speed, proof size, and hardware scalability. We then give a brief overview of each component. For more details, we refer the reader to section 6.

- *Field.* The impact of the field on the above-mentioned performance metrics mainly depends on its bit length and compatibility with 32- or 64-bit hardware registers. A larger bit width allows the field to represent wider numerical ranges and support more complex or larger circuits, but it also increases memory consumption and arithmetic cost. Fields whose modulus matches the hardware word size allow modular reduction to be done more efficiently. In particular, the Goldilocks field is carefully designed to fit 64-bit registers while supporting fast modular reduction and compact memory layout, achieving both lower memory usage and faster computation.
- *PIOP.* PIOP defines how computations are turned into algebraic constraints and how the prover and verifier interact during proof generation. Its design directly affects proof size, prover speed, and memory consumption. Fewer interaction rounds and simpler constraint structures can reduce proving time and memory use, while more complex formulations provide stronger expressiveness but higher cost. The PIOP design, therefore, determines how efficiently the system encodes and verifies computations in zkVM provers.
- *PCS.* PCS defines how polynomials are committed and verified, which directly impacts proof size, prover speed, and verification cost. Efficient schemes [20, 26] can reduce the number of openings and simplify verification, while more complex constructions achieve smaller proofs but require heavier computation. For example, IPA- and KZG-based schemes [19, 53] offer compact proofs with fast verification based on elliptic-curve operations. In contrast, FRI-based schemes [25, 27] avoid this assumption and support recursion more naturally, at the cost of larger proof size and higher communication overhead. Therefore, the choice of

PCS defines the balance between proof size, proving efficiency, and verification scalability in zkVM provers.

**Security Considerations.** The security of zkVMs depends critically on both the underlying cryptographic primitives and implementation correctness. Different PCS families carry distinct trust assumptions: KZG requires a trusted setup ceremony, while hash-based PCS like FRI are transparent. Recent work [54] shows that small-field hash-based proof systems may have weaker soundness than previously conjectured, requiring careful parameter selection. Beyond cryptographic soundness, implementation bugs in constraint systems—particularly in memory checking and instruction encoding—represent significant attack vectors [17]. We provide a comprehensive security analysis covering soundness vulnerabilities, cryptographic assumptions, and formal verification requirements in Appendix B.

## 4 Instruction Set Architecture

ISA determines how high-level programs are translated into low-level operations that can be further arithmetized and proven. The ISA directly affects the execution trace structure, algebraic state transition, and the efficiency of the proof layer.

### 4.1 ISA Taxonomy

Modern zkVMs adopt three main ISA paradigms: general-purpose ISAs, zk-friendly ISAs, and hybrid ISAs. Each paradigm reflects a particular stance on the trade-off between programmability and algebraic regularity.

General-purpose ISAs, such as RISC-V [50] and MIPS [52], start from traditional processor design and reuse decades of compiler and tooling infrastructure. zkVMs like Risc0 [7], SP1 [9], OpenVM [13], and Ceno [30] execute binaries compiled for standard RISC-V targets and therefore inherit mature language support, optimization passes, and debugging workflows. This choice significantly lowers the adoption barrier: existing Rust or C++ code can be moved into a zkVM setting with minimal rewriting, and developers can retain familiar toolchains. At the same time, the semantics of such ISAs were never designed with polynomial IOPs [24] in mind. Bitwise operations, arithmetic instructions, flag logic, and heterogeneous instruction formats all translate into irregular constraint systems. The resulting execution traces tend to be wide, heterogeneous, and dominated by small, low-level state transitions that must all be verified in the proof.

Compared with general-purpose ISAs, zk-friendly ISAs take the opposite path. Systems like Cairo [37, 38], Valida [41] begin from the algebraic model and design instructions that map cleanly to field operations and low-degree polynomial relations. Instruction formats are uniform, the register model is minimal, and memory access follows patterns that are easy to encode via permutation arguments [5] or lookups [28]. This leads to the execution traces with highly regular structure and substantially lower constraint density per unit of useful computation. However, this algebraic regularity comes at the price of ecosystem maturity. zk-friendly ISAs typically require new languages or domain-specific compilers, offer limited library support, and impose unfamiliar programming models. Porting existing software stacks is non-trivial, and developers must reason directly about the constraints induced by the ISA.

Hybrid ISAs absorb the advantages from both general-purpose ISAs and zk-friendly ISAs. In such designs, the base ISA remains general-purpose, most commonly RISC-V, but is extended with specialized instructions or call interfaces that route high-cost operations to dedicated circuits. Systems like SP1 [9, 10] and Risc0 [7, 8] preserve the ability to compile arbitrary programs written in high-level programming languages to targeted binary code. When the guest program reaches such an entry point, the VM delegates the computation to a specialized subcircuit. This subcircuit is designed to minimize the number of constraints required to implement the operation. In practice, hybrid ISAs keep the programming experience close to a conventional CPU while ensuring that the cryptographic hotspots dominating real workloads do not explode proof size or proving time.

### 4.2 ISA Structure and Trace Geometry

Beyond the high-level paradigm, the internal structure of an ISA, including granularity, register model, control-flow mechanisms, and memory semantics, plays a decisive role in shaping the geometry of the execution trace. These design choices directly influence the complexity of the corresponding VM circuits.

Instruction granularity is one of the most visible dimensions. General-purpose ISAs typically decompose high-level operations into instructions including additions, bitwise operations, shifts, branches, and loads and stores. Each of these instructions produces a separate row in the execution trace, all of them belong to a single logical step. The result is a large number of trace rows and a dense schedule of small state transitions that must be constrained. In contrast, zk-friendly ISAs tend to define instructions at a more algebraic granularity. An instruction may update multiple state components at once in a way that can be captured by a small fixed set of polynomial identities, or it may encode domain-specific operations that map directly to low-degree relations. This difference in granularity is identical to a difference in the number of rows per unit of computation.

The register and state model has a similar effect. A 32-bit register architecture such as RV32I (32-bit RISC-V ISA) offers flexible temporary storage and supports aggressive compiler optimizations, but every register must be represented and updated in the trace. For zkVMs, this means that each cycle includes a wide vector of register values, and VM circuits must ensure that reads and writes are consistent across time, often with permutation arguments over large tables. zk-friendly ISAs such as Cairo [38] reduce the number of architectural registers to a handful, using memory to store most intermediate values and relying on simple pointer-like registers for navigation. This reduction in architectural state collapses the width of each trace row and can drastically reduce the number of constraints needed to enforce register consistency.

Memory semantics have a strong influence on how the VM is structured. When an ISA uses a unified Von Neumann memory model, instructions and data share one address space and can be accessed in arbitrary patterns. This forces the VM to maintain a global memory-consistency argument that handles all types of

accesses together. In contrast, ISAs that separate program memory from data memory, or that limit how memory can be updated, allow the VM to break the memory-checking task into smaller and more regular components. The VM is free to choose its own memory-checking mechanism, but the constraints it must enforce are still largely determined by the memory model defined by the ISA.

Taken together, these structural choices determine the trace geometry: the number of rows assigned to each logical operation, the number and type of columns included in each row, and the constraints that link these rows. ISAs that promote regular and algebraically simple traces tend to produce cleaner constraint systems. ISAs that introduce wide, bit-level, or irregular state transitions instead require more complex arithmetization and heavier proving machinery.

### 4.3 Cryptographic and Specialized Operations

Cryptographic and other domain-specific operations account for a substantial portion of the cost in many zkVM workloads. Tasks such as hash evaluations, Merkle tree updates, signature verification, and pairing-based computations appear frequently in blockchain and privacy-preserving applications. The way an ISA represents these operations determines whether they must be expanded into long sequences of general-purpose instructions or can instead be delegated to dedicated algebraic components.

In a general-purpose ISA, cryptographic primitives are implemented as ordinary machine code. A single hash function can expand into hundreds or even thousands of bit-level instructions such as rotations, shifts, and Boolean operations, and each instruction adds one row to the execution trace. For the prover, this style of computation is expensive because it often requires breaking field elements into bits or using large lookup tables to capture bitwise behavior over a prime field. As a result, an algorithm that appears compact in source form ends up producing a large trace segment and becomes a major contributor to the overall constraint cost.

zk-friendly and hybrid ISAs attempt to avoid this explosion by giving cryptographic or arithmetic-intensive operations a more privileged representation. In zk-friendly designs, certain primitives are built into the instruction set or the language runtime. For example, instead of expressing a hash function as a sequence of basic operations, the ISA may provide a single instruction that computes a hash over a fixed-size input and updates designated memory locations. The algebraic implementation of that instruction can then be carefully optimized as a dedicated subcircuit with a fixed constraint cost. In hybrid designs, a similar effect is achieved by reserving parts of the opcode space or syscall interface for "accelerated" primitives. Guest programs invoke these operations through a stable interface, while the underlying zkVM routes the call to an appropriate chip or accelerator circuit.

This separation establishes a clear boundary. The base ISA handles control flow and general computation, while cryptography-intensive operations are executed by specialized, constraint-efficient components. Which operations are implemented in this way depends on the application. For example, zkEVM-style systems focus on Ethereum-compatible hashing and signature schemes, while other applications may use different elliptic curves, accumulators, or vectorized arithmetic. Across systems, the same pattern is observed: ISAs that explicitly represent these operations allow the VM and prover to treat them as algebraic atoms, whereas ISAs that provide only low-level primitives require the proving system to reason about cryptography at the instruction level.

## 5 Execution Layer

The execution layer operates between the ISA and the proving layer. It runs the compiled program and produces raw execution traces. The executor has two main components. The emulator executes the program according to the ISA semantics. The execution tracer records state transitions for downstream proving. While the ISA defines instruction meaning, the execution layer determines how instructions are executed and recorded. Its design strongly affects trace size and regularity, which in turn influences constraint complexity and prover performance. This section organizes the design space of execution architectures and analyzes how execution and memory model choices affect proving efficiency.

### 5.1 Executor Architecture

The executor is the core component that bridges program semantics and trace generation. Its design determines how the ISA semantics are realized and how the resulting state transitions are captured for proof generation.

**Emulator Design Patterns.** The emulator interprets compiled programs according to the target ISA specification. Two primary design patterns have emerged in modern zkVMs. In the interpretation-based execution model, most zkVMs decode and execute instructions one at a time. The emulator maintains the architectural state, including registers, the program counter, and memory, and applies ISA-defined transformations for each instruction. This approach is flexible and simple to implement but can incur overhead from repeated instruction decoding.

In contrast, some zkVMs adopt a trace-oriented execution model, which is optimized for producing trace-friendly output rather than raw execution performance. In this model, the emulator generates traces in a format suitable for arithmetization, reducing the need for post-processing. For example, Jolt [11] uses standard interpretation for execution but captures instruction semantics via lookup tables during proving. Each instruction's input-output behavior is then verified using the Lasso lookup argument [33] instead of explicit constraint evaluation.

**Execution Tracer.** The execution tracer records state transitions during program execution, producing the information that will later be arithmetized by the proving layer. It needs to balance multiple considerations, including capturing sufficient detail for constraint verification, maintaining efficient data structures for large traces, and organizing output in formats suitable for downstream processing. In addition, modern zkVM tracers generally record the following information for each execution step: the program counter value, decoded opcode and operands, register file state for register-based ISAs, and memory access events such as address, value, and read/write type. The exact subset and encoding of this information depend on the target arithmetization format.

| Paradigm | Technique | Verification | Complexity | Parallelism | Soundness Basis | Representative zkVMs | Reference |
|---|---|---|---|---|---|---|---|
| **Online** | Merkle Tree | Per-access | $O(\log n)$/access | Low | Collision resistance | Early designs | [55] |
| | Paging + Merkle | Per-page-fault | $O(\log n)$/page | Medium | Collision resistance | RISC Zero (hybrid) | [8] |
| **Offline** | Grand Product (Perm) | Batch | $O(n)$ total | High | Schwartz-Zippel | SP1, OpenVM, Pico, Valida | [28, 55] |
| | LogUp | Batch | $O(n)$ total | High | Schwartz-Zippel | Cairo, Nexus | [38, 56] |
| | Spice/Lasso/Twist | Batch | $O(m+n)^\dagger$ | High | Sumcheck soundness | Jolt | [11, 33, 57, 58] |
| | GKR Sumcheck | Layered | $O(n \log n)$ | High | GKR soundness | Ceno | [30, 59] |
| | Chiplet-based | Batch | $O(n)$ total | Medium | AIR constraints | Miden | [40] |

**Notes:** $n$ denotes the number of memory accesses; $m$ denotes number of lookups. Complexity refers to total prover cost. $^\dagger$Lasso [33] achieves $O(m+n)$ prover cost for $m$ lookups into size-$n$ tables. The foundational work by Blum et al. [55] established both online (per-access Merkle proofs) and offline (batch multiset equality) paradigms. Hybrid approaches (e.g., RISC Zero) combine Merkle proofs for page-level access with permutation arguments for intra-page consistency.

**Table 2: Taxonomy of Memory Checking Techniques in zkVMs**

## 5.2 Memory Architecture

The memory architecture of a zkVM determines how memory accesses are organized, tracked, and verified. While Section 5 focuses on the execution-layer perspective, memory checking mechanisms that enforce consistency are detailed in Section 5.3.

**Address Space Organization.** Although zkVMs commonly emulate a unified Von Neumann memory model, where instructions and data share a single address space, their circuit-level organization often follows a Harvard-style decomposition. Instruction fetches and program-counter transitions are validated by one set of circuits, while data loads, stores, and address-value consistency are validated by another. This separation reduces constraint coupling and allows each component to be optimized independently.

**Memory Models.** Different zkVMs adopt distinct memory models according to their design philosophy. Some, such as SP1, RISC Zero, and Jolt, implement standard read-write RAM, allowing arbitrary reads and writes. Others, including Cairo and Cairo-M, use write-once memory, where each address can be written at most once, simplifying consistency checks. RISC Zero additionally supports paged memory with Merkle-based verification, while Miden and zkWasm rely primarily on a stack-based memory model where operations are performed through an operand stack. These design choices influence how memory accesses are represented in execution traces and the complexity of the resulting constraints.

## 5.3 Memory Checking Mechanisms

Memory consistency, which requires that each read returns the most recently written value at a given address, is one of the most technically challenging aspects of zkVM design. Foundational work by Blum et al. [55] introduced two paradigms, including online and offline memory checking.

**Online Memory Checking.** Online approaches verify memory consistency incrementally as the execution trace is generated. A common technique uses Merkle trees: each memory access updates a Merkle root, and the prover shows that read values match the authenticated state through inclusion proofs. While conceptually simple, online checking embeds hash computations directly into the constraint system, resulting in $O(\log n)$ overhead per access and increased circuit depth. This method was used in early zkVM designs but has largely been replaced by offline approaches for performance reasons. RISC Zero's paging mechanism [8] illustrates a hybrid strategy: memory is divided into 1KB pages, with Merkle proofs required only for the first access to a page, while later accesses within the same page are verified offline.

**Offline Memory Checking.** Offline approaches, also introduced by Blum et al. [55], are dominant in modern zkVMs. These methods record all memory operations during execution and check consistency in a separate, batched phase. The key insight is that memory consistency can be reduced to a multiset equality check: the multiset of all (address, value, timestamp) tuples from writes must match the multiset from reads, with timestamps adjusted appropriately. This reduction allows efficient batch verification using various techniques.

- **Permutation Argument (Grand Product)** [28]: Encode read/write tuples as field elements and prove multiset equality via: $\prod_i(\alpha - \text{enc}(W_i)) = \prod_j(\alpha - \text{enc}(R_j))$, where $\alpha$ is a random challenge. Used by SP1, OpenVM, Pico, Valida, and Airbender.
- **LogUp (Logarithmic Derivative)** [56]: Reduces multiset equality to sumcheck relations using logarithmic derivatives: $\sum_i \frac{m_i}{\alpha - T_i} = \sum_j \frac{1}{\alpha - Q_j}$, achieving lower constraint degree. Adopted by Cairo (via Stwo) and Nexus.
- **Spice (Timestamp-based)** [57]: Timestamp-augmented memory checking assigns each memory cell a timestamp that increments on writes, allowing reads to verify they retrieve the most recent value. Jolt [11] extends this approach with Lasso lookup arguments [33], which work with any multilinear polynomial commitment scheme and achieve prover costs of $O(m+n)$ for $m$ lookups into a table of size $n$, while structured tables require no commitment, supporting sizes up to $2^{128}$ or larger. Recent work such as Twist and Shout [58] further improves memory checking: Twist handles read/write memories, Shout targets read-only memories, and both achieve logarithmic verifier costs with prover costs over ten times lower than prior state-of-the-art methods configured for logarithmic proof length.

The choice between online and offline memory checking involves a key trade-off: online methods provide immediate validation but incur per-access overhead, while offline methods enable parallelism and batch verification at the cost of deferred error detection. Modern zkVMs favor offline approaches due to their compatibility with STARK/SNARK arithmetization, where batch verification amortizes costs effectively.

# 6 Prover Architecture

## 6.1 Overview

At a high level, the proof generation process is defined as below.

(1) **Receive and Arithmetize Trace.** The prover receives the execution trace from the execution layer and arithmetizes it according to the VM circuit definitions, transforming raw state transitions into algebraic constraints over a finite field.
(2) **Commit Trace.** The prover encodes the trace as polynomials and commits to the trace using a PCS.
(3) **Compute and Commit Prover Polynomials.** The prover applies the polynomial IOP protocol to generate prover polynomials, and then commit to these polynomials with PCS.
(4) **Open Polynomials.** The prover uses the PCS opening to generate evaluations along with proofs at random challenges. The random challenges are generated by a random oracle.

## 6.2 VM Circuits and Arithmetization

Once the trace is defined, the prover must encode correctness through a modular constraint system. Instruction semantics, memory consistency, and auxiliary invariants are distributed across specialized circuits or "chips," each responsible for enforcing a particular aspect of program behavior. These circuits collectively ensure that every transition in the trace follows the ISA-defined semantics.

Most zkVMs share a common set of circuit components that collectively capture the semantics of the ISA. A CPU-style circuit enforces opcode decoding, register updates, flag transitions, and program-counter evolution. A memory circuit enforces read/write consistency, ensuring that each load observes the most recent store. Additional circuits specialize in arithmetic-heavy or control-heavy opcode classes, exploiting lookup arguments, non-native arithmetic, or algebraically tailored constraints to reduce degree and improve performance.

Some VMs isolate cryptographic primitives—such as hash functions or signature verification—into dedicated circuits that use highly optimized gadgets or precomputed tables. Others include circuits for range checks, padding rules, address canonicalization, or checkpoint consistency. The degree of circuit decomposition reflects the VM's design philosophy: finer granularity improves modularity and allows targeted optimizations, whereas coarser granularity simplifies integration but increases per-circuit complexity.

## 6.3 Recursive Proof Architecture

To scale to large programs, zkVMs rarely generate one monolithic proof. Instead, they adopt a recursive architecture shown in Figure 3: the global trace is split into segments, each segment is proven independently, and the resulting proofs are aggregated through one or more layers of recursive verifiers. The final root proof attests to the correctness of the entire execution and can optionally be wrapped in a succinct SNARK. This architecture enables parallelism, modular verification, incremental proving, and flexible deployment across diverse proof backends.

The segmentation strategy—its boundaries, chunk size, and interface conditions—directly influences recursion depth and proof aggregation cost. The proving architecture therefore plays a central role in determining whether recursive composition remains shallow and predictable or becomes irregular and expensive.

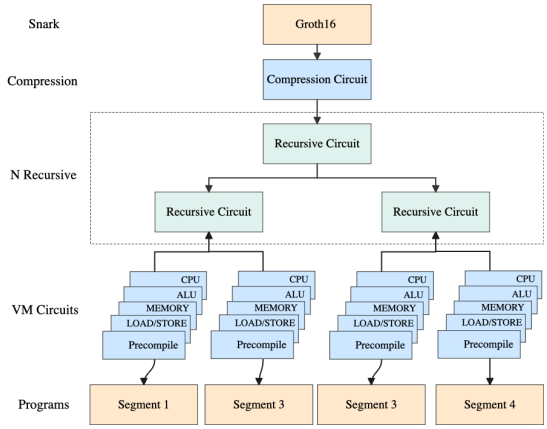


**Figure 3: Recursive proof aggregation architecture in modern zkVMs. The execution trace is segmented and proven by specialized leaf circuits, then recursively aggregated through multiple layers to produce a single root proof.**

## 6.4 Prover Algorithm

The performance of zkVM provers is typically evaluated using a small set of core metrics:

(1) **Constraint throughput.** Measures the number of constraints processed per second throughout the proving pipeline, including trace generation, commitment, and proof construction. This metric captures the raw scalability and computational efficiency of the prover.
(2) **Proof size and verification latency.** Reflect the deployability of the system, especially in recursive and on-chain verification settings. Smaller proofs and lower latency reduce on-chain costs and improve composability for large zkVM workloads.
(3) **Memory footprint during FFT/MSM/Merkle phases.** Represents the peak memory consumption during FFTs, MSMs, and Merkle tree computations. It often determines the maximum circuit size that a prover can handle efficiently in practice.

These metrics are determined by the design choices of the proof system, including the field that defines the arithmetic environment, the PIOP that determines constraint encoding and interaction pattern, and the PCS that governs how commitments and openings are performed. Different combinations of these components lead to distinct trade-offs between prover speed, proof size, and hardware scalability. We then outline these building blocks.

**Field.** The choice of finite field determines the arithmetic domain, directly affecting prover time and verification efficiency. Table 4 summarizes common fields. Small fields (BabyBear, Goldilocks) enable fast SIMD/GPU operations but require degree-4 extensions for 128-bit security, adding 9–16× overhead per extension operation. Large pairing-friendly fields (BN254) avoid extensions but incur slower base arithmetic. Mersenne31 ($2^{31} - 1$) offers extremely fast

| PCS Type | Setup | Proof Size | Verifier Cost | Prover Cost | Recursion | Transparency | Security |
|---|---|---|---|---|---|---|---|
| **KZG (Pairing-based)** | Trusted (SRS) | Constant ($O(1)$) | Very low (1–2 pairings) | Low–Moderate (MSM) | Low | ✗ | Bilinear (non-PQ) |
| **IPA (Inner-product)** | Transparent | Logarithmic ($O(\log n)$) | Moderate (multi-MSM) | Moderate | High | ✓ | Discrete log (non-PQ) |
| **Code-based** | Transparent | Large ($\tilde{O}(\sqrt{n})$) | Moderate (hash checks) | Near-linear | High | ✓ | Hash / IT (PQ) |
| **FRI (Hash-based)** | Transparent | Large (logarithmic) | Low–Moderate (hash ops) | High (FFT + hash) | Very high | ✓ | Hash-based (post-quantum) |

✓: supported, ✗: unsupported, PQ = post-quantum.

**Table 3: Polynomial Commitment Schemes in zkVM Provers**

reduction and is adopted in Circle STARKs [60] for FFT over the circle group.

| Field | Modulus $p$ | Bit Length | Key Advantages |
|---|---|---|---|
| **BabyBear** | $2^{31} - 2^{27} + 1$ | 31-bit | Fits 32-bit registers. Fast SIMD/GPU parallelism. |
| **Goldilocks** | $2^{64} - 2^{32} + 1$ | 64-bit | 2-adic friendly. FFT-friendly on 64-bit CPUs. |
| **Mersenne31** | $2^{31} - 1$ | 31-bit | Extremely fast reduction. Used in Circle STARK. |
| **BN254** | 254-bit prime | 254-bit | Pairing-friendly. Slower for FFT. |

**Table 4: Common Finite Fields in zkVM Provers**

**PIOP.** Modern zkVM provers can be broadly categorized into three major architectures: sumcheck-based, AIR/STARK-based, and PLONKish-based systems. Sumcheck-based provers, such as Jolt [11, 33] and Ceno [30], leverage the sumcheck protocol for efficient lookup arguments (e.g., Lasso) and multilinear polynomial evaluations. This design supports high parallelism and provides low verifier complexity, making it well-suited for GPU acceleration. AIR/STARK-based provers, such as RISC Zero and Cairo, encode execution traces as Algebraic Intermediate Representations (AIR) and use FRI-based [25] proximity proofs for verification. Meanwhile, PLONKish provers, such as SP1, follow a polynomial IOP approach with PLONKish constraint systems and FRI-based commitments. They encode execution traces as low-degree polynomials, leveraging lookups and custom gates to reduce proof size and support recursion. In addition, hybrid approaches have been proposed to combine the advantages of sumcheck-based and PLONKish frameworks, integrating the efficient parallel proof generation of sumcheck with the flexible polynomial commitment and recursion capabilities of PLONKish systems.

*(1) Sumcheck-based Protocols.* The sumcheck protocol [61] verifies the sum of a multivariate polynomial over a Boolean hypercube. Jolt [11] and Ceno [30] build upon sumcheck with Lasso [33] lookups, achieving $O(m)$ prover cost for $m$ lookups into tables of size up to $2^{128}$. The GKR protocol [59, 62] extends sumcheck for layered circuits with quasi-linear prover and logarithmic verifier complexity [63, 64]. Key advantages for zkVM: (1) instruction decoding maps naturally to multilinear extensions; (2) round-by-round structure enables fine-grained GPU parallelization.

*(2) AIR/STARK and PLONKish Systems.* AIR-based STARKs (RISC Zero, Cairo) encode traces as polynomial constraints that vanish on a trace domain; PLONKish systems (SP1, Plonky3 [65]) add custom gates and copy constraints [5]. The trace is encoded as wire polynomials over a subgroup $H$, and correctness reduces to verifying that a quotient polynomial $Q(x) = C(x)/Z_H(x)$ has low degree via FRI [25]. PLONKish systems enforce wiring through permutation grand products. Both use FRI for transparent commitments and support lookup arguments [28, 33] and recursion [48]. The constraint degree $d$ creates a trade-off: higher $d$ reduces rows but increases FFT domain size; degree-3 to degree-5 is typical.

*(3) Hybrid.* Emerging hybrid approaches combine sumcheck-based proofs with FRI-based commitments [25, 65], exploiting sumcheck's parallelism for local computation and FRI's recursion-friendliness for aggregation [48].

**PCS.** PCS defines how polynomials are committed and opened, directly influencing proof size, verifier cost, and recursion capability. Table 3 summarizes the four major families:

*(1) KZG [19].*: Constant-size proofs via pairings; best succinctness but requires trusted setup and difficult recursion (pairing verification is expensive in-circuit).

*(2) IPA.* Transparent with $O(\log n)$ proof size; recursion-friendly as it avoids pairings, used in Halo2 and Spartan [6].

*(3)Code-based [64, 66, 67].* Transparent and post-quantum secure via error-correcting codes; larger proofs but near-linear prover time.

*(4) FRI [25].* Hash-based, transparent, post-quantum; excellent GPU parallelism and recursion-friendly. Dominant in modern zkVMs (RISC Zero, SP1) despite larger proof sizes.

# 7 Benchmark and Implementation

## 7.1 Experimental Setup

We evaluate a set of zkVM systems that expose different stages of the proving pipeline and report their performance under a unified benchmark environment.

**zkVM Systems.** Our benchmark covers zkVMs drawn from all major design families identified in our taxonomy, including general-purpose CPU-style systems such as RISC0 and SP1, zk-friendly ISA designs such as Ziren and Pico, and hybrid or algebraic-VM architectures such as Airbender, Cairo-M, Jolt, Miden, and Nexus. Among these systems, RISC0, SP1, Ziren, and Pico support end-to-end VM-circuit proving and recursive compression, although Pico does not provide Groth16 APIs for SNARK wrapping. Airbender, Cairo-M, Jolt, Miden, and Nexus expose only the VM-circuit proving stage and do not implement compression or Groth16 proof generation. These capability differences determine which stages appear in our measurements and directly explain the missing entries labeled "–" in Table 5.

**Benchmarks and Metrics.** We use three programs, including Fibonacci, Hash, and Signature, to exercise arithmetic loops, hashing, and elliptic-curve operations.[1] For each program and input $n$, we

[1] Our benchmarks are designed as micro-benchmarks to isolate and compare architectural characteristics across zkVMs, rather than to simulate production-scale workloads. Real-world applications such as zkEVM block execution or large-scale Merkle tree

| Program | zkVM | Input | Execution | | | VM Circuits | | Compression | | Groth16 | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | | | Instr. | Time (s) | Cycles | Time (s) | Size (B) | Time (s) | Size (B) | Time (s) | Size (B) | Verify (s) |
| Fibonacci | RISC0 | 50 | 4,088 | – | 32,768 | 1.56 | 209,256 | 6.84 | 222,668 | 9.17 | 256 | 0.003 |
| | RISC0 | 100 | 4,438 | – | 32,768 | 1.55 | 209,256 | 6.85 | 222,668 | 9.08 | 256 | 0.003 |
| | SP1 | 50 | 5,293 | 0.004 | 5,293 | 3.60 | 1,799,745 | 4.04 | 1,314,741 | 7.47 | 260 | 0.002 |
| | SP1 | 100 | 5,643 | 0.003 | 5,643 | 3.50 | 1,799,745 | 3.98 | 1,314,741 | 7.41 | 260 | 0.002 |
| | Ziren | 50 | 5,027 | 0.016 | 5,027 | 2.35 | 2,590,175 | 5.91 | 1,109,739 | 7.45 | 260 | 0.002 |
| | Ziren | 100 | 5,477 | 0.017 | 5,477 | 2.35 | 2,590,175 | 6.06 | 1,109,739 | 7.72 | 260 | 0.002 |
| | Pico | 50 | – | 0.020 | 5,361 | 1.99 | 2,916,221 | 34.04 | 321,064 | – | – | – |
| | Pico | 100 | – | 0.020 | 5,361 | 2.08 | 2,916,221 | 33.32 | 321,064 | – | – | – |
| | Airbender | 50 | – | 0.014 | – | 106.76 | 8,260,561 | – | – | – | – | – |
| | Airbender | 100 | – | 0.005 | – | 106.35 | 8,260,561 | – | – | – | – | – |
| | Cairo-M | 50 | – | 0.000 | – | 1.10 | 902,444 | – | – | – | – | – |
| | Cairo-M | 100 | – | 0.000 | – | 0.78 | 919,124 | – | – | – | – | – |
| | Jolt | 50 | – | – | – | 1.50 | 70,690 | – | – | – | – | – |
| | Jolt | 100 | – | – | – | 1.51 | 70,690 | – | – | – | – | – |
| | Miden | 50 | – | 0.001 | 2,048 | 0.09 | 51,142 | – | – | – | – | – |
| | Miden | 100 | – | 0.001 | 4,096 | 0.14 | 57,693 | – | – | – | – | – |
| | Nexus | 50 | 26,258 | 0.199 | – | 2.39 | 56,240 | – | – | – | – | – |
| | Nexus | 100 | 26,258 | 0.198 | – | 2.39 | 55,088 | – | – | – | – | – |
| Hash | RISC0 | 500 | 7,238 | – | 32,768 | 1.55 | 209,256 | 6.77 | 222,668 | 9.32 | 256 | 0.003 |
| | SP1 | 500 | 8,443 | 0.004 | 8,443 | 3.49 | 1,799,745 | 3.94 | 1,314,741 | 7.43 | 260 | 0.002 |
| | Ziren | 500 | 9,047 | 0.025 | 9,047 | 2.46 | 2,590,175 | 6.15 | 1,109,739 | 7.41 | 260 | 0.002 |
| | Pico | 500 | – | 0.023 | 5,361 | 2.07 | 2,916,221 | 32.76 | 321,064 | – | – | – |
| | Airbender | 500 | – | 0.005 | – | 102.60 | 8,260,561 | – | – | – | – | – |
| | Cairo-M | 500 | – | 0.000 | – | 0.67 | 909,032 | – | – | – | – | – |
| | Jolt | 500 | – | – | – | 1.76 | 75,154 | – | – | – | – | – |
| | Miden | 500 | – | 0.008 | 16,384 | 0.48 | 67,101 | – | – | – | – | – |
| | Nexus | 500 | 26,258 | 0.194 | – | 2.38 | 54,464 | – | – | – | – | – |
| Signature | RISC0 | 1 | 3,749 | – | 32,768 | 1.54 | 209,256 | 6.81 | 222,668 | 9.14 | 256 | 0.003 |
| | RISC0 | 10 | 3,808 | – | 32,768 | 1.55 | 209,256 | 6.82 | 222,668 | 9.23 | 256 | 0.002 |
| | RISC0 | 50 | 4,088 | – | 32,768 | 1.52 | 209,256 | 6.90 | 222,668 | 9.28 | 256 | 0.003 |
| | SP1 | 1 | 4,954 | 0.003 | 4,954 | 3.50 | 1,799,745 | 3.94 | 1,314,741 | 7.50 | 260 | 0.002 |
| | SP1 | 10 | 5,013 | 0.003 | 5,013 | 3.59 | 1,799,745 | 3.96 | 1,314,741 | 7.46 | 260 | 0.002 |
| | SP1 | 50 | 5,293 | 0.004 | 5,293 | 3.48 | 1,799,745 | 3.93 | 1,314,741 | 7.57 | 260 | 0.002 |
| | Ziren | 1 | 4,452 | 0.015 | 4,452 | 2.73 | 2,527,825 | 6.00 | 1,109,739 | 7.53 | 260 | 0.002 |
| | Ziren | 10 | 4,560 | 0.016 | 4,560 | 2.41 | 2,590,175 | 6.36 | 1,109,739 | 7.53 | 260 | 0.002 |
| | Ziren | 50 | 5,027 | 0.017 | 5,027 | 2.36 | 2,590,175 | 6.04 | 1,109,739 | 7.18 | 260 | 0.002 |
| | Pico | 1 | – | 0.017 | 5,361 | 2.76 | 2,916,221 | 35.37 | 321,064 | – | – | – |
| | Pico | 10 | – | 0.021 | 5,361 | 2.38 | 2,916,221 | 34.61 | 321,064 | – | – | – |
| | Pico | 50 | – | 0.020 | 5,361 | 2.20 | 2,916,221 | 34.30 | 321,064 | – | – | – |
| | Airbender | 1 | – | 0.005 | – | 101.29 | 8,312,574 | – | – | – | – | – |
| | Airbender | 10 | – | 0.005 | – | 104.69 | 8,312,574 | – | – | – | – | – |
| | Airbender | 50 | – | 0.005 | – | 102.92 | 8,312,574 | – | – | – | – | – |
| | Cairo-M | 1 | – | 0.000 | – | 0.63 | 850,568 | – | – | – | – | – |
| | Cairo-M | 10 | – | 0.000 | – | 0.67 | 865,928 | – | – | – | – | – |
| | Cairo-M | 50 | – | 0.000 | – | 0.67 | 884,004 | – | – | – | – | – |
| | Jolt | 1 | – | – | – | 1.42 | 66,226 | – | – | – | – | – |
| | Jolt | 10 | – | – | – | 1.44 | 66,226 | – | – | – | – | – |
| | Jolt | 50 | – | – | – | 1.49 | 70,690 | – | – | – | – | – |
| | Miden | 1 | – | 0.000 | 256 | 0.01 | 39,547 | – | – | – | – | – |
| | Miden | 10 | – | 0.000 | 512 | 0.03 | 43,247 | – | – | – | – | – |
| | Miden | 50 | – | 0.001 | 2,048 | 0.08 | 51,142 | – | – | – | – | – |
| | Nexus | 1 | 26,258 | 0.196 | – | 2.37 | 55,648 | – | – | – | – | – |
| | Nexus | 10 | 26,258 | 0.195 | – | 2.37 | 55,648 | – | – | – | – | – |
| | Nexus | 50 | 26,258 | 0.195 | – | 2.39 | 56,240 | – | – | – | – | – |

**Table 5: Performance comparison of representative zkVMs on three program types (Fibonacci, Hash, Signature). Metrics include instruction count (Instr.), execution time, cycles, and proving metrics for three stages in the proving pipeline: VM-circuit proving, proof compression, and Groth16 conversion. "–" denotes that the zkVM does not expose the metric, or the corresponding phase is not supported. Pico's Groth16 results are omitted due to exceeding the device's 48 GB memory. Airbender, Cairo-M, Jolt, Miden, and Nexus currently support only VM-circuit proving.**

record: (1) execution metrics, including instruction count, cycles, execution time, (2) VM-circuit proving time and base proof size, (3) compression time and aggregated proof size if a zkVM supports, and (4) Groth16 proving time, proof size, and verification time for proof compression if a zkVM supports. In addition, a missing entry labeled as '-' in Table 5 denotes that the stage is not avilable or not exposed by the zkVM system.

**Benchmark Environment** All systems are built with their official toolchains and evaluated on a MacBook Pro equipped with an Apple M4 Pro processor (14 cores) and 48 GB RAM. To ensure that the results reflect system behavior rather than incidental fluctuations, we run each configuration ten times and report the average.

## 7.2 Performance Analysis

We analyze the benchmark results as shown in Table 5 from two perspectives: cross-system analysis and intra-system scaling analysis. The cross-system analysis focuses on performance differences among various zkVM implementations under the same workload, while the intra-system analysis examines how each zkVM scales as the input size increases. The experimental results closely match the architectural distinctions summarized in Table 1, confirming the trends predicted by our taxonomy.

*7.2.1 Cross-system analysis.* This part compares zkVMs under identical workloads in terms of execution trace size, proving throughput, and backend costs. Across all benchmarks, systems with compact trace layouts, such as SP1, Ziren, and Miden, show more consistent resource usage. Systems whose traces contain fixed overhead, such as RISC0 and Nexus, include additional cost that does not depend on program size.

**Execution Trace Size.** Trace size differs mainly because of ISA density and how each VM constructs its algebraic trace. SP1 and Ziren generate roughly 5,000 instructions for a medium-scale Fibonacci workload, which reflects the program's native control flow. RISC0 expands each instruction into several trace rows and pads all executions to a fixed trace of 32,768 cycles, which leads to a much lower ratio between instructions and cycles. Nexus produces the largest traces (about 26,000 instructions) because its fold-based execution model introduces a fixed amount of work for every program. These trends show that trace size depends more on VM architecture than on the computation itself.

**Proving Throughput.** Throughput differences follow the same architectural patterns. Miden reaches the highest throughput (23–43 kHz) because its stack-based VM matches its algebraic backend closely. RISC0 also reaches high throughput (around 21 kHz) through an optimized STARK pipeline, although its padded traces add extra cost. SP1, Ziren, and Pico fall within a moderate range (1.5–2.5 kHz). Jolt and Cairo-M generate smaller proofs (66–75 KB) but achieve lower throughput because their constraint systems rely heavily on table lookups.

**Compression and SNARK Wrapping.** Backend operations also show clear differences. During compression, RISC0 produces the smallest compressed proofs ( 222 KB), SP1 completes compression fastest ( 4 s), and Ziren provides a balanced result in both dimensions. Pico produces a compact compressed proof ( 321 KB) but requires about 34 s due to a less optimized recursive verifier. In the SNARK wrapping stage, RISC0, SP1, and Ziren generate nearly identical Groth16 proofs (256–260 bytes) with proving times around 7–9 s. Verification is consistently fast (2–3 ms). These results indicate that the SNARK stage depends only on the fixed verifier circuit and not on the program being executed.

*7.2.2 Intra-system Analysis.* This part evaluates how each zkVM responds when the input size increases. The comparison focuses on how traces grow, how proving time scales with trace size, and whether backend operations remain stable across inputs.

**Trace Growth.** SP1 and Ziren show proportional increases in both instruction count and cycle count as the input grows, which reflects the expected rise in program complexity. Miden enlarges its trace in a predictable manner as well and doubles its cycle count (2,048→4,096) when the input doubles. RISC0 hides growth at the cycle level because every program is padded to 32,768 cycles, so only the instruction count increases. Nexus shows little change because its fold-based design introduces nearly fixed overhead for every run.

**Proof Generation.** Proving time grows with trace size in all zkVMs. Proof size, however, remains almost unchanged within each system because proof size is determined by the structure of the VM circuit instead of the length of the computation. SP1 produces proofs of about 1.8 MB, Ziren produces proofs of about 2.5 MB, RISC0 produces proofs of about 209 KB, Jolt remains between 66 and 75 KB, and Nexus remains around 55 KB.

**Backend Stability.** Backend operations remain constant across all input sizes. RISC0 always compresses its base proof to roughly 222 KB in about 6.8 s and generates a 256-byte Groth16 proof in about 9 s. SP1 and Ziren show the same input-independent pattern. These results confirm that backend cost is determined entirely by the fixed verifier circuit and does not scale with the computation.

verification may involve millions of instructions and require distributed proving infrastructure, which is beyond the scope of this single-machine evaluation. Nevertheless, the relative performance trends and architectural trade-offs observed in our experiments are expected to generalize to larger workloads.

## Acknowledgement

This work was supported by the Postdoctoral Fellowship Program of CPSF (Grant No. GZB20250407), the Zhejiang Provincial Natural Science Foundation of China (Grant No. LMS26F020011), the China Postdoctoral Science Foundation (Grant No. 2025M771571), and the National Natural Science Foundation of China (Grant No. 62232002). This project was also supported by Input Output Global (IOG). We thank OpenVM and Risc Zero for their corrections of this work.

## Ethics Statements and Compliance with the Open Science Policy

**Ethics Statements.** This research does not involve any ethical concerns, as it does not include activities that could pose harm or risk to individuals or organizations. All open-source libraries can be found on GitHub. We hope these research findings can offer new insights and support to scholars and developers working in related fields.

**Open Science Policy.** We fully adhere to the principles of the Open Science Policy and are committed to promoting transparency

and reproducibility in scientific research. We publicly open-source the code of this work [2].

## References


[1] Shafi Goldwasser, Silvio Micali, and Chales Rackoff. The knowledge complexity of interactive proof-systems. In *Providing sound foundations for cryptography: On the work of shafi goldwasser and silvio micali*, pages 203–225. 2019.

[2] Amos Fiat and Adi Shamir. How to prove yourself: Practical solutions to identification and signature problems. In *Conference on the theory and application of cryptographic techniques*, pages 186–194. Springer, 1986.

[3] Eli Ben-Sasson, Alessandro Chiesa, Daniel Genkin, Eran Tromer, and Madars Virza. Snarks for c: Verifying program executions succinctly and in zero knowledge. In *Annual cryptology conference*, pages 90–108. Springer, 2013.

[4] Jens Groth. On the size of pairing-based non-interactive arguments. In *Annual international conference on the theory and applications of cryptographic techniques*, pages 305–326. Springer, 2016.

[5] Ariel Gabizon, Zachary J Williamson, and Oana Ciobotaru. Plonk: Permutations over lagrange-bases for oecumenical noninteractive arguments of knowledge. *Cryptology ePrint Archive*, 2019.

[6] Srinath Setty. Spartan: Efficient and general-purpose zksnarks without trusted setup. In *Annual International Cryptology Conference*, pages 704–737. Springer, 2020.

[7] RISC Zero. Risc zero: A zero-knowledge verifiable general computing platform. GitHub repository, 2024. https://github.com/risc0/risc0.

[8] RISC Zero Team. RISC Zero zkVM: Scalable, transparent arguments of RISC-V integrity. Whitepaper, 2023. https://www.risczero.com/proof-system-in-detail.pdf.

[9] Succinct Labs. Sp1: A performant, 100% open-source, contributor-friendly zkvm. GitHub repository, 2024. https://github.com/succinctlabs/sp1.

[10] Succinct Labs. SP1: A high-performance, open-source zkVM. Technical Documentation, 2024. https://docs.succinct.xyz/.

[11] Arasu Arun, Srinath Setty, and Justin Thaler. Jolt: Snarks for virtual machines via lookups. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, pages 3–33. Springer, 2024.

[12] Scroll Tech. Scroll: Native zkEVM layer 2 for ethereum. https://github.com/scroll-tech/scroll, 2024.

[13] Axiom. OpenVM: Modular, performant zkVM framework. https://github.com/openvm-org/openvm, 2024.

[14] Ryan Lavin, Xuekai Liu, Hardhik Mohanty, Logan Norman, Giovanni Zaarour, and Bhaskar Krishnamachari. A survey on the applications of zero-knowledge proofs. *arXiv preprint arXiv:2408.00243*, 2024.

[15] Junkai Liang, Daqi Hu, Pengfei Wu, Yunbo Yang, Qingni Shen, and Zhonghai Wu. Sok: Understanding zk-snarks: The gap between research and practice. *arXiv preprint arXiv:2502.02387*, 2025.

[16] Karel Velička. Zero-knowlege virtual machines. 2025.

[17] Christoph Hochrainer, Valentin Wüstholz, and Maria Christakis. Arguzz: Testing zkvms for soundness and completeness bugs. *arXiv preprint arXiv:2509.10819*, 2025.

[18] Thomas Gassmann, Stefanos Chaliasos, Thodoris Sotiropoulos, and Zhendong Su. Evaluating compiler optimization impacts on zkvm performance. *arXiv preprint arXiv:2508.17518*, 2025.

[19] Aniket Kate, Gregory M Zaverucha, and Ian Goldberg. Constant-size commitments to polynomials and their applications. In *International conference on the theory and application of cryptology and information security*, pages 177–194. Springer, 2010.

[20] Benedikt Bünz, Jonathan Bootle, Dan Boneh, Andrew Poelstra, Pieter Wuille, and Greg Maxwell. Bulletproofs: Short proofs for confidential transactions and more. In *2018 IEEE symposium on security and privacy (SP)*, pages 315–334. IEEE, 2018.

[21] Eli Ben-Sasson, Iddo Bentov, Yinon Horesh, and Michael Riabzev. Scalable, transparent, and post-quantum secure computational integrity. *Cryptology ePrint Archive*, 2018.

[22] Alessandro Chiesa, Yuncong Hu, Mary Maller, Pratyush Mishra, Noah Vesely, and Nicholas Ward. Marlin: Preprocessing zksnarks with universal and updatable srs. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, pages 738–768. Springer, 2020.

[23] Benedikt Bünz, Ben Fisch, and Alan Szepieniec. Transparent snarks from dark compilers. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, pages 677–706. Springer, 2020.

[24] Eli Ben-Sasson, Alessandro Chiesa, and Nicholas Spooner. Interactive oracle proofs. In *Theory of Cryptography Conference*, pages 31–60. Springer, 2016.

[25] Eli Ben-Sasson, Iddo Bentov, Yinon Horesh, and Michael Riabzev. Fast reed-solomon interactive oracle proofs of proximity. In *45th international colloquium on automata, languages, and programming (icalp 2018)*, pages 14–1. Schloss Dagstuhl–Leibniz-Zentrum fuer Informatik, 2018.

[26] Sean Bowe, Jack Grigg, and Daira Hopwood. Recursive proof composition without a trusted setup. *Cryptology ePrint Archive*, 2019.

[27] Eli Ben-Sasson, Alessandro Chiesa, Michael Riabzev, Nicholas Spooner, Madars Virza, and Nicholas P Ward. Aurora: Transparent succinct arguments for r1cs. In *Annual international conference on the theory and applications of cryptographic techniques*, pages 103–128. Springer, 2019.

[28] Ariel Gabizon and Zachary J Williamson. plookup: A simplified polynomial protocol for lookup tables. *Cryptology ePrint Archive*, 2020.

[29] Brevis Network. Pico: A lightweight zkVM. https://github.com/brevis-network/pico, 2024.

[30] Scroll Tech. Ceno: Zero-knowledge proofs for RISC-V. https://github.com/scroll-tech/ceno, 2024.

[31] Matter Labs. Airbender: STARK-based zkVM. https://github.com/matter-labs/zksync-airbender, 2024.

[32] a16z crypto research. Jolt: SNARKs for virtual machines via lookups. GitHub repository, 2024. https://github.com/a16z/jolt.

[33] Srinath Setty, Justin Thaler, and Riad Wahby. Unlocking the lookup singularity with lasso. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, pages 180–209. Springer, 2024.

[34] Nexus Inc. Nexus zkvm. GitHub repository, 2024. https://github.com/nexus-xyz/nexus-zkvm/.

[35] StarkWare. Stwo: Starkware's next-generation prover. https://github.com/starkware-libs/stwo, 2024.

[36] Polygon Labs. Zisk: RISC-V zkVM for zero-knowledge proofs. https://github.com/0xPolygonZero/zisk, 2024.

[37] StarkWare. Cairo programming language. https://github.com/starkware-libs/cairo-lang, 2024.

[38] Lior Goldberg, Shahar Papini, and Michael Riabzev. Cairo–a turing-complete stark-friendly cpu architecture. *Cryptology ePrint Archive*, 2021.

[39] KKRT Labs. Cairo-M: A micro-architecture for cairo. https://github.com/kkrt-labs/cairo-m, 2024.

[40] Polygon Labs. Miden VM: STARK-based virtual machine. https://github.com/0xPolygonMiden/miden-vm, 2024.

[41] Lita Foundation. Valida zkVM: STARK-based zero-knowledge virtual machine. https://github.com/lita-xyz/valida, 2024.

[42] ProvableHQ. snarkVM: A zero-knowledge virtual machine. https://github.com/ProvableHQ/snarkVM, 2024.

[43] Lean Ethereum. Lean zkVM: Formal verification for zkVMs. https://github.com/leanEthereum/leanMultisig, 2025.

[44] ProjectZKM. Ziren: ZKM zkVM implementation. https://github.com/ProjectZKM/Ziren, 2024.

[45] O1 Labs. o1vm: A MIPS-based zkVM. https://github.com/o1-labs/proof-systems, 2024.

[46] Delphinus Lab. zkWasm: Zero-knowledge WebAssembly virtual machine. https://github.com/DelphinusLab/zkWasm, 2024.

[47] Microsoft Research. Nova: Recursive zero-knowledge arguments. https://github.com/microsoft/nova, 2024.

[48] Abhiram Kothapalli, Srinath Setty, and Ioanna Tzialla. Nova: Recursive zero-knowledge arguments from folding schemes. In *Annual International Cryptology Conference*, pages 359–388. Springer, 2022.

[49] Powdr Labs. Powdr: Modular zero-knowledge stack. https://github.com/powdr-labs/powdr, 2024.

[50] Andrew Waterman, Yunsup Lee, David A Patterson, and Krste Asanovic. The risc-v instruction set manual, volume i: Base user-level isa. *EECS Department, UC Berkeley, Tech. Rep. UCB/EECS-2011-62*, 116:1–32, 2011.

[51] Sarah Harris and David Harris. *Digital Design and Computer Architecture, RISC-V Edition*. Morgan Kaufmann, 2021.

[52] Optimism Foundation. Cannon: MIPS-based fault proof VM. https://github.com/ethereum-optimism/optimism/tree/develop/cannon, 2024.

[53] Benedikt Bünz, Mary Maller, Pratyush Mishra, Nirvan Tyagi, and Psi Vesely. Proofs for inner pairing products and applications. In *International conference on the theory and application of cryptology and information security*, pages 65–97. Springer, 2021.

[54] Giacomo Fenzi and Antonio Sanso. Small-field hash-based snargs are less sound than conjectured. *Cryptology ePrint Archive*, 2025.

[55] Manuel Blum, Will Evans, Peter Gemmell, Sampath Kannan, and Moni Naor. Checking the correctness of memories. *Algorithmica*, 12(2):225–244, 1994.

[56] Ulrich Haböck. Multivariate lookups based on logarithmic derivatives. *Cryptology ePrint Archive*, 2022.

[57] Srinath Setty, Sebastian Angel, Trinabh Gupta, and Jonathan Lee. Proving the correct execution of concurrent services in zero-knowledge. In *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18)*, pages 339–356, 2018.

[58] Srinath Setty and Justin Thaler. Twist and shout: Faster memory checking arguments via one-hot addressing and increments. *Cryptology ePrint Archive*, 2025.


[2] https://github.com/SuccinctPaul/zkvm-benchmarks

[59] Shafi Goldwasser, Yael Tauman Kalai, and Guy N Rothblum. Delegating computation: interactive proofs for muggles. *Journal of the ACM (JACM)*, 62(4):1–64, 2015.
[60] Ulrich Haböck, David Levit, and Shahar Papini. Circle starks. *Cryptology ePrint Archive*, 2024.
[61] Carsten Lund, Lance Fortnow, Howard Karloff, and Noam Nisan. Algebraic methods for interactive proof systems. *Journal of the ACM (JACM)*, 39(4):859–868, 1992.
[62] Justin Thaler. Time-optimal interactive proofs for circuit evaluation. In *Annual cryptology conference*, pages 71–89. Springer, 2013.
[63] Tiacheng Xie, Jiaheng Zhang, Yupeng Zhang, Charalampos Papamanthou, and Dawn Song. Libra: Succinct zero-knowledge proofs with optimal prover computation. In *Annual International Cryptology Conference*, pages 733–764. Springer, 2019.
[64] Tiancheng Xie, Yupeng Zhang, and Dawn Song. Orion: Zero knowledge proof with linear prover time. In *Annual International Cryptology Conference*, pages 299–328. Springer, 2022.
[65] Polygon Project. Plonky3. https://github.com/Plonky3/Plonky3, 2024.
[66] Scott Ames, Carmit Hazay, Yuval Ishai, and Muthuramakrishnan Venkitasubramaniam. Ligero: Lightweight sublinear arguments without a trusted setup. In *Proceedings of the 2017 acm sigsac conference on computer and communications security*, pages 2087–2104, 2017.
[67] Alexander Golovnev, Jonathan Lee, Srinath Setty, Justin Thaler, and Riad S Wahby. Brakedown: Linear-time and field-agnostic snarks for r1cs. In *Annual International Cryptology Conference*, pages 193–226. Springer, 2023.
[68] Shankara Pailoor, Yanju Chen, Franklyn Wang, Clara Rodríguez, Jacob Van Geffen, Jason Morton, Michael Chu, Brian Gu, Yu Feng, and Işıl Dillig. Automated detection of under-constrained circuits in zero-knowledge proofs. *Proceedings of the ACM on Programming Languages*, 7(PLDI):1510–1532, 2023.
[69] Howard Wu, Wenting Zheng, Alessandro Chiesa, Raluca Ada Popa, and Ion Stoica. {DIZK}: A distributed zero knowledge proof system. In *27th USENIX Security Symposium (USENIX Security 18)*, pages 675–692, 2018.
[70] Huayi Qi, Minghui Xu, Xiaohua Jia, and Xiuzhen Cheng. Vdoram: Towards a random access machine with both public verifiability and distributed obliviousness. *Cryptology ePrint Archive*, 2025.

# Appendix

## A zkVM Metrics

To enable fair cross-system comparisons and reproducible research, we propose a standardized metric protocol for zkVM evaluation. This protocol defines consistent measurement points across the proving pipeline, aligned with the three-layer architecture presented in this work. Table 6 summarizes all metrics organized by pipeline phase.

**Metric Collection Guidelines.** To ensure reproducibility, we recommend the following protocols: (1) *Environment specification*: report hardware configuration (CPU model, core count, memory capacity), software versions (zkVM version, compiler toolchain), and runtime settings (thread count, optimization flags). (2) *Repetition*: execute each configuration at least 10 times and report both mean and standard deviation; discard the first run to account for cold-start effects. (3) *Missing values*: for unsupported phases, mark entries as "—" and document the reason. (4) *Baseline programs*: use standardized benchmarks (e.g., Fibonacci, hash, signature) with clearly defined input parameters.

## B Security Analysis

While previous sections focus on the architectural and performance aspects of zkVMs, security remains a fundamental concern for practical deployments. In this section, we systematically analyze the security landscape of zkVMs from three perspectives: soundness vulnerabilities, cryptographic assumptions, and implementation security considerations.

### B.1 Soundness Vulnerabilities in zkVMs

Soundness is the most critical security property of a zkVM: it guarantees that no computationally bounded adversary can produce a valid proof for a false statement. However, recent work [17] has revealed that real-world zkVMs contain soundness bugs that could allow malicious provers to forge proofs. We identify three primary sources of soundness vulnerabilities:

**Memory Consistency Bugs.** Memory checking is one of the most complex components in zkVM constraint systems. Bugs in memory consistency enforcement can allow a malicious prover to "read" values that were never written, breaking the integrity of computation. Common vulnerability patterns include:

- **Missing timestamp constraints**: Failing to enforce temporal ordering allows out-of-order reads.
- **Incomplete address range checks**: Unbounded addresses can bypass memory isolation.
- **Permutation argument errors**: Incorrect multiset construction in grand-product checks.

**Instruction Constraint Gaps.** Each ISA instruction must be fully constrained to prevent semantic violations. Incomplete constraints often arise from:

- **Edge cases**: Overflow, underflow, and division-by-zero conditions.
- **Flag register updates**: Missing carry, zero, or sign flag constraints.
- **Branch conditions**: Incomplete branch target validation.

**Lookup Table Vulnerabilities.** Lookup-based zkVMs (e.g., Jolt) rely on precomputed tables for instruction verification. Security issues include:

- **Table completeness**: Missing entries can cause false rejections or acceptances.
- **Table binding**: Weak commitment to lookup tables enables table substitution attacks.

### B.2 Cryptographic Assumptions and Security Levels

The security of a zkVM ultimately depends on the underlying cryptographic primitives. Different proof systems rely on distinct hardness assumptions, leading to varying security guarantees.

**Pairing-Based Schemes (KZG).** KZG commitments rely on the hardness of the discrete logarithm problem in pairing-friendly groups. While offering constant-size proofs and fast verification, they require a trusted setup ceremony. A compromised ceremony (e.g., leaked toxic waste) would allow universal proof forgery. Production systems like SP1 and RISC Zero use KZG for final Groth16 wrapping, inheriting this trust assumption.

**Hash-Based Schemes (FRI).** FRI-based commitments achieve transparency by relying only on collision-resistant hash functions modeled as random oracles. This provides post-quantum security but requires careful parameter selection to ensure sufficient soundness error bounds. The soundness of FRI depends on the Reed-Solomon code parameters and the number of query rounds.

**Field Size and Security Margin.** The choice of finite field directly impacts security levels. Smaller fields like BabyBear (31-bit) and Goldilocks (64-bit) offer computational efficiency but require extension fields or larger proof parameters to achieve standard security

| Phase | Metric | Symbol | Unit | Description | Key |
|---|---|---|---|---|---|
| **Execution** | Instruction Count | $I_{count}$ | instructions | Total executed instructions | `instruction_count` |
| | Total Cycles | $C_{total}$ | cycles | Total CPU cycles consumed | `total_cycles` |
| | Execution Time | $T_{exec}$ | seconds | Wall-clock execution time | `execution_time_s` |
| | Syscall Cycles | $C_{syscall}$ | cycles | Cycles spent in syscalls | `total_syscall_cycles` |
| | Touched Memory | $A_{mem}$ | addresses | Unique memory addresses accessed | `touched_memory_addresses` |
| **VM Circuit** | Chunk Count | $N_{chunk}$ | chunks | Number of trace segments | `vm_chunk_count` |
| | Chunk Size | $R_{chunk}$ | rows | Rows per chunk | `vm_chunk_size_rows` |
| | VM Proving Time | $T_{prove}$ | seconds | Time to generate base proof | `vm_prove_time_s` |
| | Base Proof Size | $S_{base}$ | bytes | Size of VM circuit proof | `vm_prove_proof_size_bytes` |
| | Throughput | $\theta_{prove}$ | kHz | Proving throughput: $C_{total}/(T_{prove} \times 1000)$ | `vm_prove_khz` |
| **Aggregation** | Recursion Layers | $D_{recur}$ | levels | Depth of recursive aggregation | `recursion_layers` |
| | Aggregation Time | $T_{agg}$ | seconds | Time for proof aggregation | `aggressive_prove_time_s` |
| | Aggregated Proof Size | $S_{agg}$ | bytes | Size after compression | `aggressive_proof_size_bytes` |
| **SNARK** | SNARK Type | — | string | Proof system (Groth16, PLONK, etc.) | `snark_type` |
| | Setup Time | $T_{setup}$ | seconds | SNARK setup/preprocessing time | `snark_setup_time_s` |
| | Witness Time | $T_{witness}$ | seconds | Witness generation time | `snark_witness_time_s` |
| | SNARK Proving Time | $T_{snark}$ | seconds | Final SNARK proof generation | `snark_proof_time_s` |
| | Final Proof Size | $S_{final}$ | bytes | On-chain proof size | `snark_proof_size_bytes` |
| | SNARK Constraints | $N_{snark}$ | constraints | Verifier circuit constraint count | `snark_constraint_count` |
| | Verification Time | $T_{verify}$ | seconds | Proof verification time | `verification_time_s` |
| | On-chain Gas | $G_{verify}$ | gas | Estimated on-chain verification cost | `on_chain_gas_estimate` |
| **General** | Total Time | $T_{e2e}$ | seconds | End-to-end pipeline latency | `total_time_s` |
| | Total Prove Time | $T_{prove}^{total}$ | seconds | $T_{prove} + T_{agg} + T_{snark}$ | `total_prove_time_s` |
| | Peak Memory | $M_{peak}$ | MB | Peak memory consumption | `peak_memory_mb` |

**Table 6: Standardized zkVM Metrics Protocol**

| PCS Type | Assumption | Post-Quantum | Trusted Setup |
|---|---|---|---|
| KZG | DLog + Pairing | ✗ | ✓(Ceremony) |
| IPA | Discrete Log | ✗ | ✗ |
| FRI | Random Oracle (Hash) | ✓ | ✗ |
| Code-based | Hash + IT Security | ✓ | ✗ |

**Table 7: Security Assumptions of Different PCS Families**

levels (e.g., 128-bit). For instance, achieving 128-bit security over a 31-bit field typically requires working over degree-4 extension fields or increasing the number of FRI query rounds.

**Small-Field Soundness Concerns.** Recent work by Fenzi and Sanso [54] reveals that hash-based SNARGs operating over small fields may have weaker soundness than previously conjectured. The security of these systems depends on two critical parameters of the underlying linear code $C$: the distance-preservation error $\varepsilon(C, \delta)$ and the list size $|\Lambda(C, \delta)|$. When using extension codes over small base fields—precisely the configuration employed by modern zkVMs using BabyBear or Mersenne31—classical combinatorial lower bounds on list-size yield stronger attacks than anticipated. This finding has significant implications for deployed systems: zkVMs operating in the "capacity regime" (where the proximity parameter $\delta$ approaches the code's minimum distance) may not achieve their claimed security levels. Practitioners should carefully reassess soundness parameters and consider more conservative margins when deploying FRI-based proof systems over small fields.

## B.3 Implementation Security Considerations

Beyond cryptographic soundness, practical zkVM deployments face several implementation-level security challenges:

**Side-Channel Attacks.** zkVM provers process sensitive witness data during proof generation. Timing variations, memory access patterns, and power consumption can potentially leak information about private inputs. While the zero-knowledge property protects against proof-based leakage, the proving process itself may be vulnerable.

**Randomness Generation.** Interactive-to-non-interactive transformations via Fiat-Shamir require high-quality randomness derived from transcript hashing. Weak or predictable randomness can compromise soundness. zkVMs must ensure that:

- Transcript construction includes all prover messages.
- Hash function outputs are uniformly distributed over the challenge space.
- No attacker-controlled inputs can bias the random challenges.

**Verifier Implementation.** On-chain verifiers (e.g., Groth16 verifiers on Ethereum) represent high-value attack targets. Implementation bugs in pairing computations, field arithmetic, or proof deserialization can lead to critical vulnerabilities. Formal verification of verifier implementations is an active area of research.

## B.4 Formal Verification Requirements

Given the complexity of zkVM implementations and the critical nature of soundness guarantees, formal verification has become

increasingly essential for production-grade systems. Unlike traditional software where bugs may cause functional errors, a soundness bug in a zkVM can lead to catastrophic security failures—allowing malicious actors to forge proofs for arbitrary false statements.

**Verification Targets.** Formal verification efforts in zkVMs typically focus on several critical components:

- **Constraint system correctness**: Proving that the arithmetic circuit faithfully encodes the intended computation semantics, ensuring no under-constrained or over-constrained variables exist.
- **Memory consistency**: Verifying that memory read/write operations satisfy the permutation arguments and temporal ordering invariants.
- **ISA semantics**: Ensuring each instruction's constraints precisely match the formal specification of the target ISA (e.g., RISC-V).
- **Cryptographic primitives**: Verifying the correctness of finite field arithmetic, elliptic curve operations, and hash function implementations.

**Verification Approaches.** Current formal verification efforts employ multiple methodologies:

- **Theorem proving**: Using proof assistants to establish mathematical proofs of correctness. This approach provides the strongest guarantees but requires significant manual effort.
- **Automated verification**: Employing SMT solvers and symbolic execution to automatically check constraint satisfiability and detect under-constrained systems.
- **Model checking**: Exhaustively verifying finite-state properties of critical protocol components.
- **Domain-specific tools**: Specialized analyzers such as Ecne [68] that detect under-constrained bugs in R1CS and Plonkish circuits.

**Challenges and Limitations.** Despite progress, formal verification of zkVMs faces significant challenges:

- **Scale**: Modern zkVMs contain millions of constraints across hundreds of instruction types, making complete verification computationally prohibitive.
- **Specification gap**: Formal verification proves implementation matches specification, but the specification itself may contain errors or ambiguities.
- **Optimization trade-offs**: Hand-optimized circuits for performance often deviate from straightforward implementations, complicating verification.
- **Evolving codebases**: Continuous updates and optimizations require re-verification, creating maintenance burdens.

**Industry Adoption.** Production zkVM deployments increasingly mandate formal verification as part of their security assurance process. Leading projects combine formal methods with extensive security audits, fuzzing campaigns [17], and bug bounty programs to achieve defense-in-depth security. We advocate that formal verification should be considered a necessary (though not sufficient) condition for deploying zkVMs in high-stakes environments such as blockchain rollups and financial applications.

**Summary.** Soundness bugs commonly arise from memory checking and instruction constraints [17]. Trust model depends on PCS choice: KZG requires trusted setup; FRI/IPA are transparent. Recent research [54] shows small-field hash-based SNARGs may have weaker soundness than conjectured, requiring careful parameter selection. Formal verification is essential for production zkVMs. Production deployments require security audits beyond theoretical soundness proofs.

## C Conclusion

This work presents a systematization of zero-knowledge virtual machines through a three-layer architecture: ISA, VM, and proving layer Our framework addresses three research questions: it defines a unified framework that breaks zkVMs into basic components (RQ1), analyzes design principles and trade-offs across layers (RQ2), and evaluates the most representative zkVMs covering the full proving pipeline (RQ3). This taxonomy provides researchers and practitioners with a clear way to understand, compare, and extend zkVM systems, and offers a common vocabulary that helps align otherwise fragmented design choices.

Our analysis reveals several key insights. (1) ISA design fundamentally determines trace geometry: instruction granularity, register models, and memory semantics shape trace width and depth, with hybrid ISAs providing a good balance between programmability and proof efficiency. This suggests that ISA decisions cannot be treated as a purely "frontend" concern, but must be co-designed with arithmetization and prover constraints. (2) Offline memory checking has become the dominant paradigm, with permutation-based and LogUp-based techniques replacing earlier Merkle-tree approaches due to superior batch verification efficiency. This shift changes where complexity resides in the pipeline, moving more structure into lookup and permutation arguments instead of tree-based commitments. (3) Prover performance depends on the combination of finite field, polynomial IOP, and polynomial commitment scheme: small fields like BabyBear and Goldilocks paired with FRI enable efficient recursion, while KZG minimizes proof size at the cost of trusted setup. Cross-layer co-design consistently outperforms isolated single-layer optimization, since optimizing one layer alone often shifts bottlenecks elsewhere and can even negate gains if downstream stages are not adjusted accordingly.

Looking forward, several promising directions remain. These include standardized interfaces and benchmark suites (Appendix A) for reproducible evaluation, modular architectures allowing independent upgrades, hardware-accelerated and distributed proving such as GPU, FPGA, ASIC, and DIZK [69], post-quantum secure PCS constructions [54], and collaborative multi-prover zkVMs for distributed privacy-preserving computation [70]. Progress in these areas will improve the scalability, security, and interoperability of zkVMs. Future studies can investigate cross-layer optimization and modular design to achieve new gains in performance and usability across the entire stack of zero-knowledge proof.

Received 20 February 2007; revised 12 March 2009; accepted 5 June 2009